

Deliverable 1

Filippo Marcon - 268173 - filippo.marcon@studenti.unitn.it - GitHub: <https://github.com/marcoff181/spmv>

Abstract—This report evaluates the performance of GPU-based Sparse Matrix-Vector multiplication (SpMV) kernels across the CSR and COO storage formats. Through experiments on NVIDIA hardware, it compares different kernel implementations analyzing the impact of memory coalescing, load balancing and storage format.

I. INTRODUCTION

Sparse Matrix Vector multiplication (SpMV) is a fundamental problem in many computing fields. It is highly parallelizable and its performance is bounded by memory bandwidth. Formally, SpMV is defined as the multiplication between A , a sparse matrix of size $m \times n$ and x , a dense vector of size n . The resulting vector y is of size m .

$$y = Ax \quad (1)$$

This investigation focuses on relatively basic, non-adaptive GPU SpMV kernels, both for CSR and COO. We address these questions:

- What is the penalty of the additional storage requirements of COO?
- Is this penalty balanced by the ability to split work based on non-zeros without additional preprocessing?
- How much does memory coalescing improve the performance of the kernels?
- What are the ideal launch parameters for each of the kernels?
- How does matrix size, non-zeros per row, structure affect the performance of the kernels?
- How do these kernels compare to the CuSparse library performance?
- What is the memory/cache usage of our kernels?

II. METHODOLOGY

The sparse formats used in the study are CSR and COO. COO is an obvious starting point as it's the simplest storage format for sparse matrices, and it's used in the SuiteSparse dataset(see Section III.A). COO uses three arrays to store the matrix: for the n -th non-zero element of the matrix `rows[n]` indicates the row where it is located, `cols[n]` indicates the column, and `values[n]` stores the actual value. CSR still uses three arrays, with the only change being that the `rows` array is compressed with a prefix sum. The compression assumes that the non-zero elements are sorted by row index. The CSR format takes less storage than COO, however CSR-based SpMV algorithms that split tasks by row can suffer from load imbalance on sparse matrices with irregular nnz distribution along rows[1].

The CPU SpMV implementation is a simple iteration over all the non-zero elements. Its only goal is to provide a reference result to measure the error of the GPU implementations.

```
if tid < nnz:
    atomicAdd(y[rows[tid]], vals[tid] *
x[cols[tid]])
```

Listing 1. COO flat kernel

Moving to the GPU kernels, in the following pseudocode listings we assume that `tid = blockIdx.x * blockDim.x + threadIdx.x`.

One of the simplest ways to parallelize SpMV with COO is to use one thread per non-zero. This algorithm(Listing 1), commonly called *COO_flat*, assigns one thread for each non-zero element, then uses atomic adds to write the result to y . This approach offers maximum parallelism and load balancing, and a certain degree of coalescing, assuming that the COO is sorted correspondingly. On the other hand the `atomicAdd` hurts the performance a lot, and with a high enough nnz we are not able to execute all threads together.

As an improvement to the shortcomings of the flat COO kernel we propose in Listing 2 an adaptation of the COO algorithm described by Bell et al.[2]. Computation is divided in *chunks*, which can span multiple rows, each chunk has a warp(32 threads) assigned to it. The threads stride over the length of the chunk together, to guarantee memory coalescing. On each step the threads first calculate the multiplication for their assigned non-zero and then they perform a *segmented reduction* to aggregate the results. The aggregated value is carried along through the `carry` register until a new row is found, then one thread writes the aggregated result to memory. This method drastically reduces the conflicts caused by `atomicAdd`, while guaranteeing memory coalescing and balanced distribution of non-zeros. The biggest difference from the algorithm presented by Bell et al. is the usage of the more modern `shfl` instructions for the segmented reduction instead of using shared memory.

Moving to CSR format, we start with the *CSR scalar* kernel, which assigns one thread per matrix row(see Listing 3). The thread then sequentially computes the multiplication between the entire row and the corresponding value of x and finally writes it back to memory. This basic approach suffers from load imbalance proportionally to the variance of non-zeros per row of the matrix. It also has no memory coalescing, and using only one thread per row is an underutilization of GPU potential, unless the matrix is very large and with a very small amount of non-zeros per row.

To solve most of the issues of the scalar kernel we implement the *CSR_vector* kernel, which assigns a warp per matrix row. Now similarly to *COO_segmented* we have memory coalescing thanks to the threads striding together along the row. We also have a similar reduction, which is made simpler by the fact that we are not spanning rows, so we can simply use `shfl` to aggregate the 32 results and then make thread 0 write them to memory. The problem that remains unsolved is

```

carry_row ← -1
carry_val ← 0

for each tile of 32 nonzeros starting at n in
[chunk_start, chunk_end):

    Each lane i loads nonzero (row[n+i], val[n+i] *
x[col[n+i]])
    Inactive lanes (beyond chunk_end) load row = -1,
val = 0

    // --- Merge with carry from previous tile
    Lane 0 checks its row against carry_row:
    if row == carry_row:
        val ← val + carry_val
    else:
        atomicAdd(y[carry_row], carry_val)

    // --- segmented prefix sum (via shuffle)
    for offset = 1, 2, 4, 8, 16:
        left_row, left_val ← values from lane (i -
offset)
        if left_row == row:
            val ← val + left_val

    // --- Write completed rows, save carry
    next_row ← row of lane (i + 1)
    last_lane ← last active lane in tile

    if lane == last_lane:
        carry_row, carry_val ← row, val
    else if row ≠ next_row:
        atomicAdd(y[row], val)

    carry_row, carry_val ← values held by last_lane
(warp broadcast)

// --- Flush final carry
Lane 0: atomicAdd(y[carry_row], carry_val)

```

Listing 2. COO segmented reduction kernel

that we are still splitting by row, which makes the performance vulnerable to matrices with high variance of non-zeros per row.

A. Validation method

First the result y_c is calculated using the CPU algorithm, then for each execution of a kernel the result y_k is compared by dividing the l2 norm of the difference by the l2 norm of the reference. For all methods the error does not reach magnitudes over 10^{-7} .

```

if tid < num_rows:
    sum ← 0
    for each nonzero j in [rows[tid], rows[tid+1]):
        sum ← sum + vals[j] * x[cols[j]]
    y[tid] ← sum

```

Listing 3. CSR scalar kernel

```

if row < num_rows:
    sum ← 0
    for each nonzero j in [rows[row] + lane,
rows[row+1]) with stride 32:
        sum ← sum + vals[j] * x[cols[j]]

    for offset = 16, 8, 4, 2, 1:
        sum ← sum + value from lane (i + offset)

    if lane == 0:
        y[row] ← sum

```

Listing 4. CSR vector kernel

$$\text{err} = \frac{\|y_c - y_k\|_2}{\|y_c\|_2} \quad (2)$$

B. Measurement methodology

Each combination of kernel and launch parameters(block and grid size) is benchmarked by first launching the kernel WARMUP times without logging the results, and then by running it another NITER times, then computing the average of each measurement across runs. The parameters WARMUP, NITER are user-defined, and were set respectively to 2 and 10 during all the experiments.

Timings are measured through Cuda Events, and only the kernel execution time is measured. FLOP/s are measured by dividing the number of required floating point operations ($\text{nnz} * 2$) by the arithmetic mean of the execution time across the NITER runs. Bandwidth is measured by calculating total space occupied by the arrays rows, cols, values, x, y for COO and for CSR, and dividing by the arithmetic mean of the execution time.

C. Hardware/Software environment

The experiments were run on the unitn *Baldo* cluster on a NVIDIA A30 24GB graphic card. The program was compiled with gcc version 13.3.0 and cuda version 12.5.0. The code is publicly available at this link: <https://github.com/marcoff181/spmv>.

III. DATASET

A. Sparse Matrices

The dataset chosen for benchmarking the various kernels is a subset of the 14 matrices selected by S.Williams et al.[3] and later also used in a NVIDIA technical report[4], hosted on the SuiteSparse Matrix Collection[5]. This small selection consists of matrices derived from real-world problems in different fields. The matrices are intentionally varied in dimension, non-zeros per row, existence of dense block structure, and degree of non-zero concentration along the diagonal. Table I provides a summary of the dataset. CV is the *coefficient of variance* across rows calculated by dividing the variance by the mean.

TABLE I
SUMMARY OF MATRIX SELECTION

Matrix	Rows	nnz	Avg NNZ per Row	CV NNZ per Row
cant.mtx	62451	4007383	64.1684	0.2191
consph.mtx	83334	6010480	72.1252	0.2645
cop20k_A.mtx	121192	2624331	21.6543	0.6369
mac_econ_fwd500.mtx	206500	1273389	6.1665	0.7193
mc2depi.mtx	525825	2100225	3.9942	0.0191
pdb1HYS.mtx	36417	4344765	119.3060	0.2670
rma10.mtx	46835	2374001	50.6886	0.5481
scircuit.mtx	170998	958936	5.6079	0.7832
shipsec1.mtx	140874	7813404	55.4638	0.1997
webbase-1M.mtx	1000005	3105536	3.1055	8.1613

B. Parsing

The .mtx format used by SuiteSparse stores matrices in COO, with column-major order. The parsing is done

through the library `fast-matrix-market`, as the `.mtx` format allows a number of different representations to save space (standard, symmetric, skew-symmetric) meaning that writing a parser from scratch could lead to errors and/or distract from the main goal. After parsing we sort the matrix in row-major order, and then create a new vector with the rows in CSR format. The result is that we use row-major ordering for both COO and CSR, for consistence and ease of use when writing and comparing kernels.

C. Input Vector

The `Float32` Input Vector is randomly generated with a fixed seed to guarantee reproducibility across runs. The user-defined parameter `MAX_VECTOR_VALUE` defines the upper bound to the randomly generated values.

IV. RESULTS

Fig. 1, Fig. 2 show the average bandwidth and GFLOP/s of each kernel across all 10 matrices, grouped by the experimented block size. Fig. 3 shows the best kernel runtime for each matrix, matrices are sorted by total nnz.

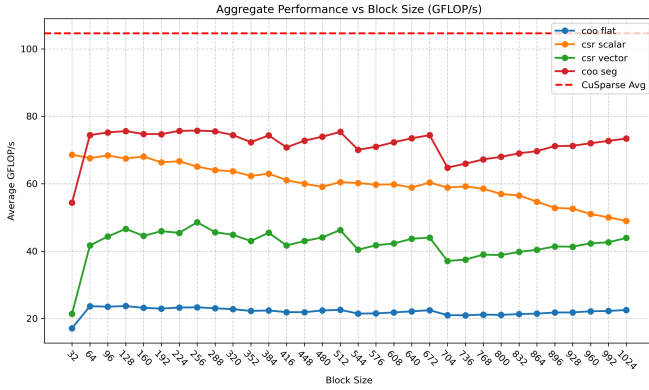


Fig. 1. Aggregate GFLOP/s related to Block Size

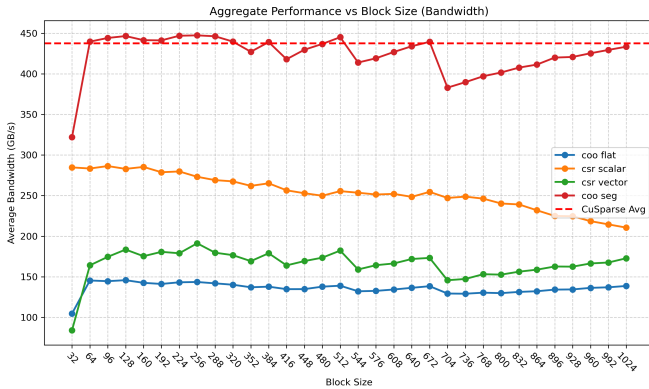


Fig. 2. Aggregate Bandwidth related to Block Size

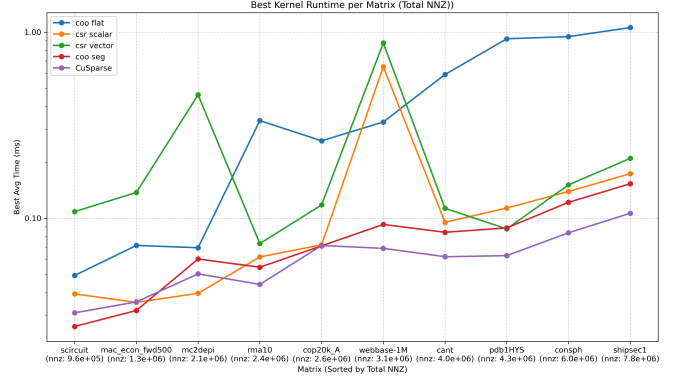


Fig. 3. Kernel best runtime for each matrix, sorted by total nnz

We briefly attempted to measure cache usage by choosing a fixed block size, and profiling one kernel at a time with `ncu` but found out that the unitn *Baldo* cluster did not allow access to NVIDIA GPU performance counters.

V. DISCUSSION

The NVIDIA A30 supports at most 32 resident blocks per SM, and has 56 SMs. This means that when setting blocksize to 32 we have $32 \times 32 = 1024$ threads per SM, while the A30 supports up to 2048 threads per SM, which is reached with block sizes of 64 and above. With this information we can explain the performance drop at blocksize 32 in Fig. 1 and Fig. 2 as being caused by not being at full occupancy. Because SpMV is highly memory-bound, having 50% occupancy results in the SM running out of active warps to switch to while the rest of the warps are waiting for the memory access.

A. CSR scalar

CSR scalar is the kernel that allocates the least amount of threads (m) out of all the kernels. For a smaller matrix such as *pdb1HYS* it allocates just $\approx 37k$ threads. The max warps per SM of the NVIDIA A30 are 64, which means that the amount of threads needed to reach full occupancy is $32 * 64 * 56 = 114688$. The decreasing performance of *CSR scalar* when increasing block size in Fig. 1 is caused by the increasing granularity in work distribution:

- with matrices that have a high variance of `nnz/row` different blocks have different memory requirements based on how many nonzero they need to load in all their assigned rows. If the blocks are bigger, memory requirements of different blocks can vary more, and the effect of scheduling a “heavy” block on one SM becomes more noticeable, as it will stall execution when the other SMs are already finished. With small block sizes, this issue is mostly mitigated by the block scheduler.
- Another compounding issue is that on smaller matrices like *pdb1HYS* with a high enough block size we aren’t even able to occupy all SMs, further penalizing the bandwidth.

Ultimately, *CSR scalar* does not coalesce memory access, as threads within the same warp access different rows of the matrix, further hurting performance.

B. CSR vector

CSR vector allocates 32 times more threads than *CSR scalar*, reducing occupancy concerns, and most importantly adopts memory coalescing inside the warp, where the 32 threads access the row with a stride. Most of the dataset matrices don't have rows long enough to see the full benefit of the coalescing, and we see a big performance penalty with matrices that have on average less than 32 non-zeros per row (see Table I, Fig. 3). This is caused by the fact that with less than 32 nonzeros some threads never do any work, as they are not assigned to a non-zero. We can see that around ≈ 60 non-zero per row the performance matches *CSR scalar*. With *pdB1HYS* we have enough non-zero per row to balance the overhead of the shuffle reduction and see the benefits of memory coalescing, outperforming *CSR scalar* (which on the contrary is penalized by the long rows).

C. COO flat

COO flat has a reasonable performance only when working on matrices that are both:

- with a low number of nonzeros per row: less conflict on the atomicAdd
- highly irregular and with high variance: it benefits from the inherent load balancing of assigning one thread per nonzero

It also achieves memory coalescing on the rows, cols and values arrays (if COO is sorted by row such as in our test). Nonetheless, the overhead of thread scheduling and especially of the atomicAdd impacts the final performance considerably.

D. COO seg

This is an example of an algorithm that fully uses the advantage of COO to have workload splitting by non-zero, which is properly balanced, unlike row-based splitting. The combination of balanced workload splitting, drastically reduced use of atomicAdd compared to *COO flat* and memory coalescing on the rows, cols, and vals arrays during the computation of the chunk makes this algorithm the best performing out of the 4 kernels, on this particular dataset. More balanced CSR algorithms (such as the one presented by Chu et al. [6]) that employ different splitting techniques have higher bandwidth potential thanks to the lower footprint of the CSR format, but it's interesting to see a COO algorithm performing so close to CuSparse (on this restricted dataset), especially as .mtx files are already stored in COO, which avoids any precomputing overhead.

E. Webbase-1M

On such a restricted dataset this matrix is really useful to understand row balancing issues in CSR. *Webbase-1M* is a web connectivity graph and the nnz per row follows a power law (most pages have a few links, pages like google have orders of magnitudes more links). We can see this with the Coefficient of Variance metric in Table I. The long rows slow down drastically the CSR kernels that don't employ any work separation inside the single row: most threads finish immediately while a few amount of threads have to compute thousands of non-zeros. COO based methods are instead

unaffected by this distribution thanks to the nonzero-based splitting.

VI. CONCLUSION

One key takeaway is the importance of dataset choice when benchmarking these kinds of algorithms that are very dependent on problem structure. Having to choose just 10 matrices was necessary to keep execution times faster and allowing fast iteration, while complete articles on the topic can afford to benchmark their algorithms on the entire SuiteSparse dataset. Correctly choosing 10 matrices that properly challenge all the kernels is not a trivial task, especially if it's done at the beginning of the research work. *Webbase-1M* is a good example of a matrix that helps highlight weaknesses in certain kernels, and without it one could see Fig. 3 and be inclined to think that *CSR scalar* performs as well as *COO seg*. On the more practical side, setting up a standardized way to launch and benchmark different kernel+block size configurations was very useful during the implementation of the kernels. Another useful detail to lessen delay when running on the cluster was making a `sbatch` request for any kind of GPU, and then letting the job compile the code matching the right architecture before running it. This reduced wait times for job scheduling to nearly zero.

REFERENCES

- [1] J. Gao, B. Liu, W. Ji, and H. Huang, "A Systematic Literature Survey of Sparse Matrix-Vector Multiplication." [Online]. Available: <https://arxiv.org/abs/2404.06047>
- [2] N. Bell and M. Garland, "Implementing sparse matrix-vector multiplication on throughput-oriented processors," in *Proceedings of the conference on high performance computing networking, storage and analysis*, 2009, pp. 1–11.
- [3] S. Williams, L. Oliker, R. Vuduc, J. Shalf, K. Yelick, and J. Demmel, "Optimization of Sparse Matrix-Vector Multiplication on Emerging Multicore Platforms," *Parallel Computing*, vol. 35, no. 3, pp. 178–194, Mar. 2009.
- [4] N. Bell and M. Garland, "Efficient Sparse Matrix-Vector Multiplication on CUDA," NVIDIA Technical Report NVR-2008-4, Dec. 2008.
- [5] T. A. Davis and Y. Hu, "The university of Florida sparse matrix collection," *ACM Trans. Math. Softw.*, vol. 38, no. 1, Dec. 2011, [Online]. Available: <https://doi.org/10.1145/2049662.2049663>
- [6] C. Genshen et al., "Efficient Algorithm Design of Optimizing SpMV on GPU," 2023, pp. 115–128.